

CHAS Malmö Utvecklingsbyrå - Energy Saver Deluxe

Sammanfattning

Energy Saver Deluxe är ett verktyg som hjälper användaren att ta informerade beslut angående elförbrukning i en fastighet med solceller och batterier. Det uppnås genom att data angående kommande väder och spotpriser för el hämtas från internet och körs genom en algoritm som räknar ut per kvart hur effektiva solceller kommer vara och hur dyr eller billig elen kommer vara. Därefter beräknas hur användaren kan spara eller tjäna pengar på sin elproduktion och -förbrukning. Resultatet är ett schema med rekommenderade handlingar för varje kvart, där en siffra anger hur stark rekommendationen är för att köpa, sälja, lagra, och använda ström.

Syftet med systemet är initialt att hanteras i enbart en fastighet, men vi ser möjligheter att expandera funktionalitet om projektet blir lyckat.

Programmets liv

När programmet startar så skapas flera processer som använder vår fristående HTTP Request Service för att hämta data om elpriser samt väder. Inställningar för hemsidor att hämta ifrån, samt exempelvis daglig tidsstämpel eller tidsintervall för återkommande hämtningar, läses från en JSON-fil. Den hämtade datan sparas ner i JSON-format och därefter sover processerna tills nästa gång de ska hämta, eller tills huvudprocessen talar om för dem att stänga ner sig.

När elpriser och väderdata finns tillgängliga går programmet igenom de filerna och gör beräkningar baserade på de parametrar som vårt program behöver för att kalkylera analysen. Mer ingående information kring analysen återfinns längre ner i dokumentet.

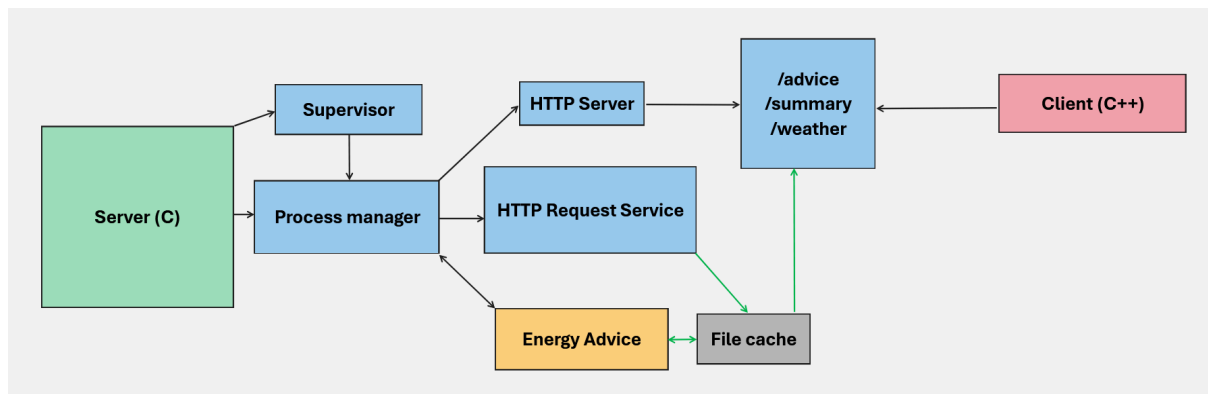
Resultatet av beräkningarna och rekommendationerna för att köpa, använda och sälja ström sammanfattas och skrivs ut i en textfil där användaren kan analysera datan själv, eller helt enkelt utgå från den automatiska rekommendationen för att göra sitt val.

Det finns tre sätt att komma åt den slutgiltiga rekommendationen. Ett alternativ är att helt enkelt öppna och läsa filen som skapas inne i projektmappen.

Programmet startar även en server man kommer åt genom localhost med en hemsida där man kan klicka sig in på filen.

Slutligen har vi en CLI-klient skriven i C++ som hämtar texten och skriver ut den i terminalen. Det finns ett alternativ för en kort sammanfattning för att undvika att överfylla terminalen.

Teknisk specifikation



Skripten i C99 och klient skripten i C++17.

HTTP Request Service som är skripten i C99 och en version i C++17 (är en äldre version).

Daemon skripten i C99.

Processer kommunicerar genom pipes samt systemanrop.

Problem och lösningar

Vi valde att använda processer istället för trådar. En anledning var för att undvika att krascha hela programmet om en tråd kraschar. Det lät oss även slippa oroa oss för misstag med data races och synkronisering.

HTTP Request Service kommunicerar med föräldraprocessen via pipes för att signalera att ny data finns. Föräldern läser från alla sina barn och om någon av dem har signalerat så kör den vår Energy Advisor som kollar så att alla filer som behövs finns och därefter tar fram det slutgiltiga dokumentet med rekommendationer och sparar ner det i en textfil.

I vår HTTP Request Service, vilken tar in en URL samt argument för återkommande hämtning, hade vi först en enkel lösning där den kallade sleep med längden tagen från argumenten. Men vi ville kunna hämta varje kvart för att inte riskera att vänta till precis i slutet av en kvart, samt kunna ange en specifik tid att hämta vid varje dag. Så vi byggde ut funktionaliteten och såg exempelvis till att den gjorde korrekta uträkningar för att rulla över till nästa dag, etc.

Energy Advisor är den del av programmet som ansvarar för algoritmen, samt analysen av de mest fördelaktiga tidpunkterna för att ladda, konsumera och sälja el. Komponenten har genomgått flera iterationer under kursens gång.

Initialt levererade systemet ett statiskt råd per kvart - att sälja, köpa, ladda eller avvakta.

Att enbart ge ett statiskt råd, när det faktiskt kan vara fördelaktigt att göra två saker samtidigt, blev problematiskt och innebar en chans för oss att strukturera om denna delen av programmet.

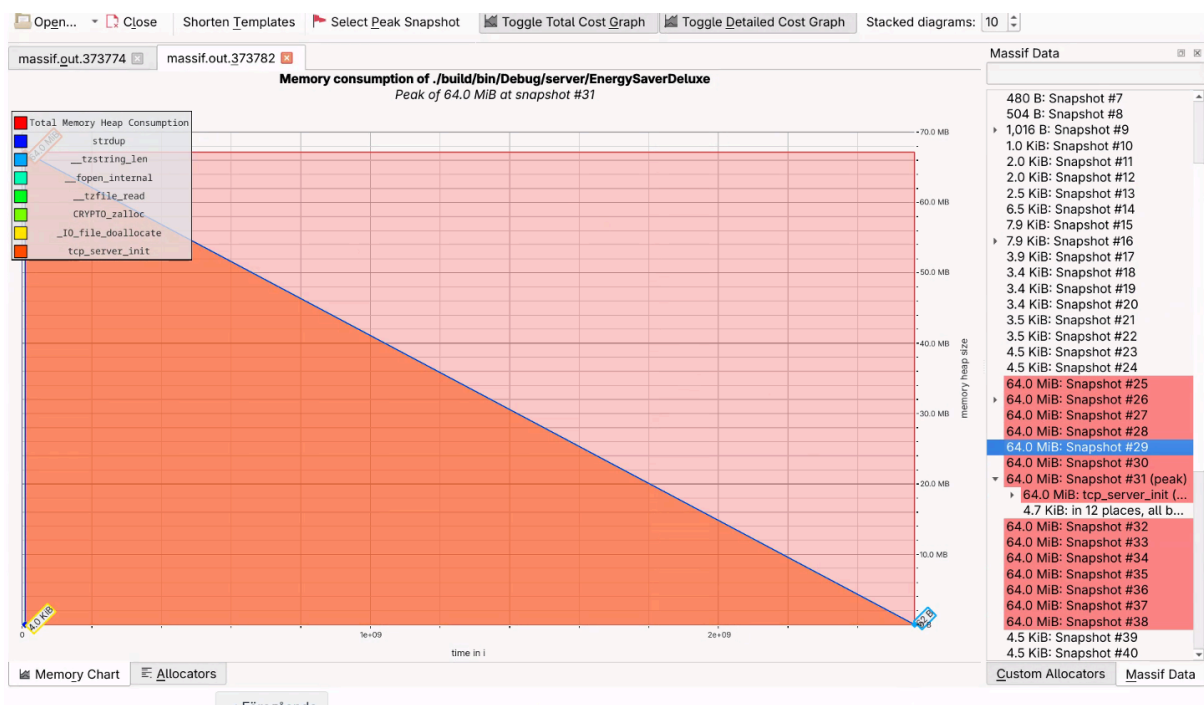
Detta har vi successivt utvecklat till en mer avancerad modell där varje kvart istället utvärderas utifrån sju parametrar, fördelade över tre huvudkategorier.

Systemet har även utökats geografiskt. Istället för att enbart baseras på en enskild plats, hämtas nu information i var och en av Sveriges fyra el-zoner. Detta möjliggör jämförelser och tydliggör regionala variationer i analyserna.

Sammanfattningsvis har Energy Advisor utvecklats från en relativt enkel rådgivare till ett mer sofistikerat och mer avancerat system. Genom att väga flera olika parametrar mot användarens definierade gränsvärden, kan programmet generera mer nyanserade och välgrundade rekommendationer för hur elförbrukningen bör hanteras på daglig basis.

När vi försökte implementera att skicka "stora mängder" data (31 KB) så fick vi en silent segmentation fault i vår TCP library. Med hjälp av GDB hittade vi att vi skapade 64mb på stacken vilket var för stort för stackramet som ledde till att vi inte ens fick segfault error. Vi löste det då genom att temporärt skapa 64mb på heapen istället vilket löste problemet.

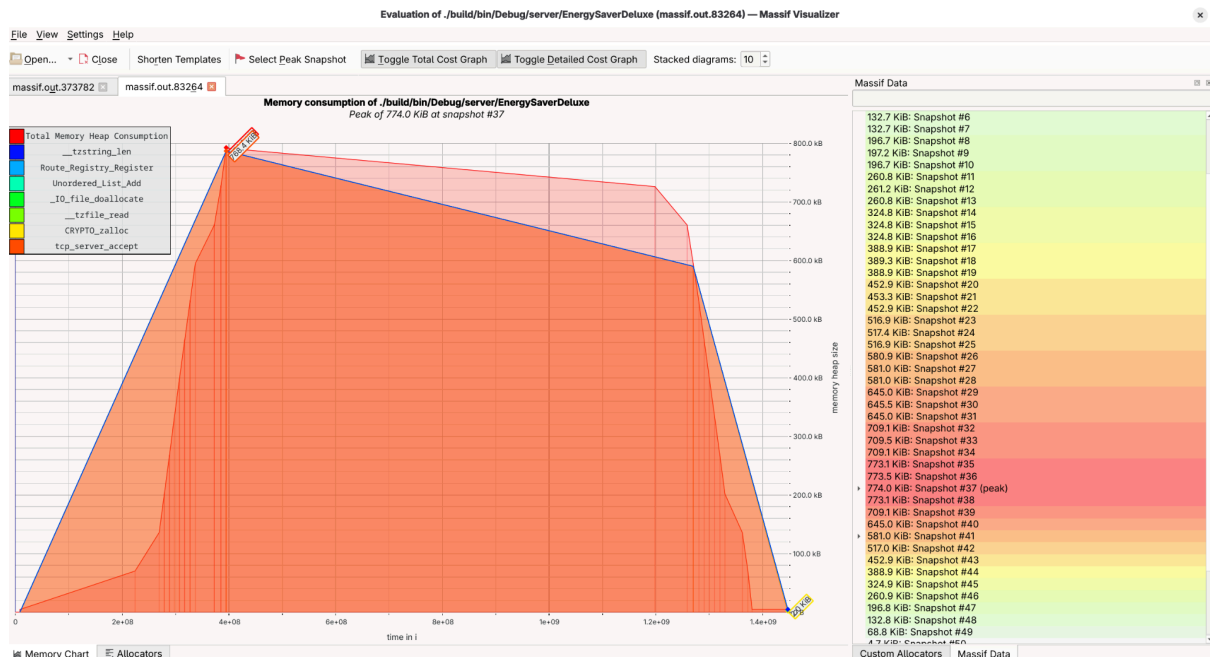
Optimeringar



Figur 1
Konstant 64MB heap användning

Genom en snabb analys av vårt program så ser vi även att vår förra temporära fix skapade en stor heap användning (den största i servern). Vi valde då att skapa en generisk Unordered List datastruktur som kunde hjälpa oss att hantera våra klienter på ett smidigare sätt, för att bidra till en mindre heap konsumtion. Datastrukturen har som värst $O(n)$, och som bäst $O(1)$. Hur datastrukturen fungerar är att den är en struct som innehåller elements, capacity och data. Datan är en void pekare som innehåller adresser av minne på antingen stack eller heap, beroende på vad utvecklaren vill göra med elementen. Det är alltid utvecklaren som måste sedan freea datan i elementen, men inte själva listan. När vi addar data till listan så antingen läggs de till ifall capacity inte är full eller så dubblar den capacity och sedan reallocar datan som finns. Sedan minskar vi aldrig kapaciteten, precis som `std::vector` gör i C++, och det är för att inte skapa overhead med att använda realloc hela

tiden, och så kan det komplicera datastrukturen onödigt mycket. Våra ändringar ledde till att memory användningen minskade drastiskt i vårt program. Se figur 2.



Figur 2

Memory användning minskas till <773KB

I vår algoritmräkning använder vi qsort för att sortera en stor array med floats. Så då valde vi att benchmarka den med vår benchmark library som vi hade skapat som är C motsvarande av chrono i C++. Den använder `clock_gettime()` vilket är en POSIX funktion och kan få som minst nanosekunder.

Värden för qsort benchmarkade då till: **3.91 μ s**(medelvärdet), vilket vi ville optimera så gott vi kunde.

Det ledde till att vi hittade radix sort algoritmen som är en non comparator sorting algoritm vilket betyder att den inte använder några comparator-operatorer. Utan använder bit shifting.

```
for (size_t i = 0; i < n; i++)
{
    uint32_t x = keys[i];
    if (x & 0x80000000) // Kollar ifall talet är negativt
        keys[i] = ~x; // Här inverterar vi alla bitar
    else
        keys[i] = x ^ 0x80000000; // Här flippar vi SIGN biten
}
```

Det första vi gör då är att omvandla alla floats till `uint32_t`, för att floats gör det svårare att sortera dem korrekt. Då ifall talet är negativt så inventerar vi talets sign bit och ifall det är ett positivt tal så flippar vi bara sign bit.

```
for (int shift = 0; shift < 32; shift += 8)
{
    // 256 för att 2^8 är 256. (0-255)
    size_t count[256] = {0};

    for (size_t i = 0; i < n; i++)
        count[(keys[i] >> shift) & 0xFF]++;
    // vi räknar ihop alla byte values i sina egna indexar
    // alla som har 1 i value tex går i sin egna index, osv.

    size_t sum = 0;

    // konverterar byte values till vart dem ska vara i arrayen
    for (int i = 0; i < 256; i++) // loopar genom 0-255
    {
        size_t tmp_sum = count[i];
        count[i] = sum;
        sum += tmp_sum;
    }

    // den faktiska sorteringen
    for (size_t i = 0; i < n; i++)
    {
        uint32_t k = keys[i];
        tmp[count[(k >> shift) & 0xFF]++] = k;
    }

    memcpy(keys, tmp, n * sizeof(uint32_t));
}
```

Efter det går vi till vår sorteringsloop, där vi loopar 8 bits i taget. (totalt 4 gånger, då vi kör `uint32_t`).

Sen i loopfunktionen kollar vi hur många gånger byte valuen av den specifika byten dyker upp och då plussas det på i count arrayen. Sedan loopar vi genom alla values och sätter value platsen. Denna loopen asså bestämmer vart någonstans i arrayen som valuet ska sorteras någonstans. Efter den loopen kommer vår sista loop, där den faktiska sorteringen händer.

```
for (size_t i = 0; i < n; i++) {  
    uint32_t x = keys[i];  
    if (x & 0x80000000)  
        keys[i] = x ^ 0x80000000;  
    else  
        keys[i] = ~x;  
}
```

Sista delen är att vi gör samma sak som vi gjorde innan fast vi inverterar det tillbaka hur det var innan för att få våra floats.

Efter att vi bytte ut qsort med en radix sort-algoritm fick vi värden då till: **2.98 µs** (medelvärde), vilket är en 24% förbättring för prestanda.

RAII - Implementation

Implementerat grundläggande klasser i vår klient som är skapad i C++. Klasser för chatbot, keyboard, endpoints och menu där vi kör med idiomatisk C++ som hanterar sina egna resurser enligt RAI-principen. Vi använder vektorer, lambdas, C++-funktionspekare bland annat.

Analys av data för algoritmuträkning

Algoritmen räknar ut solindex för att kunna räkna ut solcellsproduktion, den kontrollerar ett dygns spotpriser (96 kvartar), jämför dessa och skapar ett normaliserat värde för den givna dagen. I nuläget använder sig programmet av hårdkodad information kring användarens batteri, att det konstant är halvt fullt för att kunna generera någorlunda konsekvent data. Analys-delen av programmet tar sedan den insamlade datan och jämför dessa enligt våra sju förbestämda beslutsparametrar för att:
ladda (från elnätet, från solcellerna),
konsumera (direkt från elnätet, från solcellerna eller från batteriet),
och sälja (från batteriet eller solcellerna) och graderar dessa i ett spann mellan 0-1, där 0 inte rekommenderas medan 1 är ett absolut toppvärde och rekommenderas starkt under den givna kvarten.

Efter att den graderat alla värden så räknar den ut ett "best window" baserat på användarens preferenser (just nu satt till minst 4 sammanhängande kvartar) och jämför det mot ett gränsvärde för att se om kraven uppfylls.

Gränsvärdet beräknas dynamiskt genom att systemet kollar igenom våra värderingar inom de 3 olika beslutsdelarna och jämför det mot ett värde i den övre fjärdedelen av varje beslutsdel och på så sätt kan systemet alltid ge ett råd baserat på de faktiska värdena under dagen.

Hittas det inget sammanhängande fönster med minst 4 kvartar, så förmedlas det också i rapporten tydligt för användaren.

Framtida förbättringar/förändringar

Unix Domain Sockets var också en sak vi funderade ett tag på att implementera, men vi valde till slut att inte lägga till det, men vi hade diskussioner om att vi kanske skulle haft med dem om vi påbörjat detta projektet nu istället, då det är det snabbaste valet rent prestandamässigt, då det helt undviker kopiering av data, även fast det också kommer med risker gällande synkronisering bland annat.

Vi har börjat se ett betydligt större värde i att skapa egna funktionsvarianter av funktioner som redan finns, som t.ex. qsort för att spara en del resurser och samtidigt ge oss mer kontroll över exakt det vi vill få ut av funktionen.

Sättet användaren gör inställningar är i dagsläget ganska otympligt, för en färdig produkt hade man nog velat ha ett grafiskt gränssnitt istället för att behöva manuellt lägga in länkar och argument i en JSON-fil vilket gör det lätt att göra misstag.

Gruppmedlemmar

Emilio Ganibegovic - Tech Lead

Pär Lundh - Scrum Master

Henrik Westerlund - Utvecklare

Lukas Ståde - Utvecklare

Isa Shipshani - Utvecklare